



# **UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA**



## **Facultad de Ingeniería**

### **Ingeniería en Ciencias de la Computación**

#### **Computo Paralelo y Distribuido**

##### **Reporte de Proyecto**

Grupo: 8CC2

Oscar J. Marquez Ortega 367726  
Marco A. Romero Diaz 360927  
César E. Batista García 361432  
Diego J. Ovalle Reynoso

**Chihuahua, Chih., Mayo 2026**

## Introducción

Este proyecto desarrolla una plataforma de análisis de accidentes de tránsito en México usando datos oficiales del período 1997–2024. El objetivo es transformar 28 años de registros en bruto en un dashboard interactivo que permita explorar tendencias por estado, hora del día y causa del accidente.

El sistema está dividido en tres etapas bien separadas: procesamiento de datos, servicio de datos, y visualización. Cada etapa es independiente y puede entenderse por sí sola.

## Datos

La fuente son los registros ATUS (Accidentes de Tránsito en Zonas Urbanas y Suburbanas) del INEGI, almacenados en 28 archivos Parquet, uno por año, con un peso total aproximado de 2.5 GB. Solo el año 2023 contiene alrededor de 390,000 registros. Cada registro representa un accidente individual e incluye campos como estado, municipio, hora, mes, causa del accidente, y el desglose de víctimas por tipo (conductor, pasajero, peatón, ciclista).

Un problema encontrado durante el desarrollo es que los archivos de 1997 a 2021 tienen un defecto estructural: la primera columna (COBERTURA) fue guardada como índice del DataFrame, desplazando todas las demás columnas una posición a la derecha. Esto se detecta porque la columna ID\_MUNICIPIO termina conteniendo valores de año en lugar de claves de municipio. El script `fix_parquet.py` corrige esto automáticamente antes de que el ETL procese los datos.

## Procesamiento (ETL con Ray)

El pipeline de procesamiento está implementado en `etl_ray.py` y usa Ray para distribuir el trabajo entre múltiples procesos. Los 28 archivos se dividen en 3 lotes y cada lote se procesa en paralelo por un worker independiente.

Cada worker, por cada año que le toca, ejecuta los siguientes pasos a través de `limpieza.py`:

- Estandariza los nombres de columnas a mayúsculas (el schema varía entre años)
- Convierte las 12 columnas de víctimas a valores numéricos
- Calcula dos columnas derivadas: `TOTAL_MUERTOS` y `TOTAL_HERIDOS`, sumando las columnas individuales por tipo de víctima
- Elimina registros donde `ID_ENTIDAD` sea nulo, ya que sin estado el registro no es utilizable

Luego calcula tres agregaciones por año: totales por estado (accidentes, muertos, heridos, causa principal y tasa de mortalidad), distribución por hora del día, y el top 10 de causas globales. El resultado de todos los workers se combina, se ordena por año y se guarda en un único archivo JSON que alimenta al resto del sistema.

## API

La API está construida con FastAPI y expone dos endpoints:

- GET /api/anios — devuelve la lista de años disponibles
- GET /api/dashboard/{anio} — devuelve todos los datos procesados de un año: estados con sus métricas, distribución por hora y causas principales

Al arrancar, la API carga el JSON completo en memoria, lo que hace que las respuestas sean prácticamente instantáneas. También enriquece los datos de cada estado con su nombre oficial (el dataset solo tiene el ID numérico del 1 al 32).

## Dashboard

El dashboard es un archivo HTML estático que no requiere instalación ni build. Usa Leaflet.js para el mapa y Chart.js para las gráficas. Al seleccionar un año con el selector, consulta la API y actualiza:

- Un mapa coroplético de México con intensidad de color según el número de accidentes por estado
- Una gráfica de líneas con la distribución de accidentes por hora del día
- Una gráfica de barras con las 10 causas de accidente más frecuentes
- Una tabla con el ranking de estados por accidentes, víctimas y tasa de mortalidad

Al hacer clic en cualquier estado del mapa se despliega un panel con el detalle de esa entidad: total de accidentes, muertos, heridos y su causa principal de accidente.

## Cómo se usa

### Requisito previo

Los archivos de datos deben estar en la carpeta `data/parquet_fixed/` con el nombre `atus_anual_YYYY.parquet`. Si los archivos originales están en `data/parquet/`, hay que ejecutarlos primero por `fix_parquet.py` para corregir el defecto de columnas descrito anteriormente.

### Levantar el sistema

Con Docker instalado, basta un solo comando desde la carpeta del proyecto:

```
docker-compose up --build
```

Esto levanta automáticamente los cinco contenedores que conforman el sistema:

| Contenedor | Función |
|------------|---------|
|------------|---------|

|                  |                                              |
|------------------|----------------------------------------------|
| ray-head         | Nodo coordinador de Ray + ejecuta etl_ray.py |
| ray-worker-1/2/3 | Nodos de procesamiento distribuido           |
| api-fastapi      | Servidor FastAPI en el puerto 8000           |

El ETL arranca automáticamente y procesa los 28 años en paralelo. Dependiendo del hardware, esto puede tomar entre 5 y 15 minutos. Se puede seguir el progreso en tiempo real con:

```
docker-compose logs -f ray-head
```

## Ver el dashboard

Una vez que en los logs aparece el mensaje de ETL completado, abrir el archivo index.html directamente en el navegador (doble clic es suficiente, no se necesita servidor web). El dashboard se conecta a la API en <http://localhost:8000> y carga los datos automáticamente.

Desde ahí el uso es completamente visual: se elige el año en el selector de la barra lateral y todas las visualizaciones se actualizan al instante. En la vista de mapa también se puede cambiar la métrica que determina el color de los estados (accidentes totales, muertos, heridos o tasa de mortalidad).

## Apagar

```
docker-compose down
```

## Conclusiones

El proyecto logra su objetivo: procesar un volumen significativo de datos históricos y exponerlos en una interfaz accesible. La separación entre ETL, API y frontend es limpia y facilita modificar cualquier capa sin afectar las otras.

El uso de Ray añade paralelismo real al procesamiento, aunque el diseño actual (3 lotes secuenciales dentro de cada worker) deja margen para optimizarse si los tiempos de procesamiento fueran un factor crítico. Para el alcance del proyecto, el rendimiento es más que suficiente.